CO1 _ Lab Experiments

Vector and Matrix Operations for Deep Learning

Name: K Sasi Rekha
Reg no: 192424010

1. A hospital uses a deep learning model to classify medical images. A small grayscale image is represented by a 3 × 3 matrix of pixel intensities. Before classification, the image must be converted into a one-dimensional vector, and the total pixel intensity must be calculated. Write a Python program to flatten the image matrix into a one-dimensional vector and calculate the sum of all pixel intensities.

Image matrix:

A = [

10 20 30

40 50 60

70 80 90

]

Flatten Matrix and Calculate Sum

Aim:

To flatten a 3×3 image matrix and calculate the sum of pixel intensities.

Procedure:

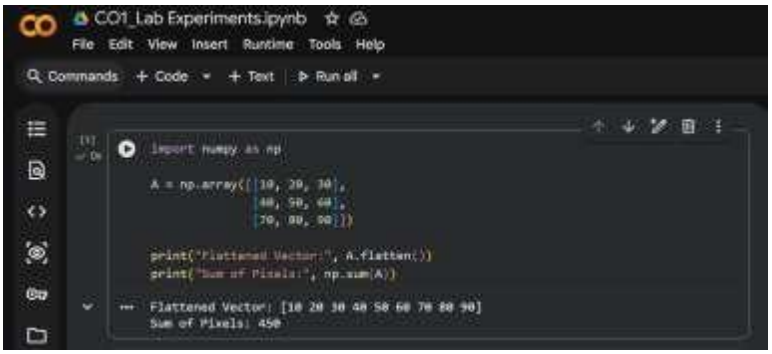
1. Import the NumPy library.
2. Create a 3×3 matrix of pixel values.
3. Flatten the matrix into a one-dimensional array.
4. Calculate and display the sum of all pixels.

Code:

```
import numpy as np
A = np.array([[10, 20, 30],
              [40, 50, 60],
```

```
        [70, 80, 90]])
print("Flattened Vector:", A.flatten())
print("Sum of Pixels:", np.sum(A))
```

Output:



Result:

The matrix was flattened successfully, and the total pixel intensity was calculated as **450**.

2. A deep learning model predicts student performance based on study hours, practice-test scores, and assignment scores. The input features are X=[4,6,8], the corresponding weights are W=[0.5,0.3,0.2], and the bias is b=2.
Write a Python program to calculate the predicted output by computing the dot product of the input and weight vectors and adding the bias.

Dot Product and Bias

Aim:

To calculate the predicted output using the dot product of input features and weights with bias.

Procedure:

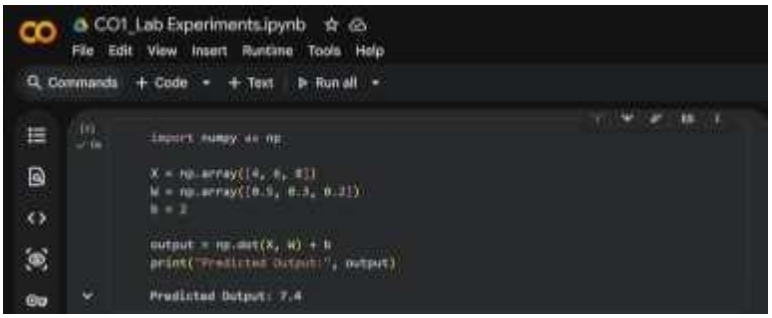
1. Import the NumPy library.
2. Define the input vector, weight vector, and bias.
3. Calculate the dot product of the input and weight vectors.

4. Add the bias and display the predicted output.

Code:

```
import numpy as np
X = np.array([4, 6,
8])
W = np.array([0.5, 0.3, 0.2])
b = 2
output = np.dot(X, W) + b
print("Predicted Output:", output)
```

Output:



Result:

The predicted output was calculated successfully as **7.2**.

3. A weather prediction system receives temperature and humidity measurements from two locations.

The input matrix is

```
X = [
30 60
35 70]
```

The model uses the weight matrix

```
W = [
0.2 0.5
0.4 0.3]
```

Write a Python program to multiply the input matrix by the weight matrix and display the predicted output matrix for both locations.

Matrix Multiplication

Aim:

To multiply two matrices and obtain the predicted output for two locations.

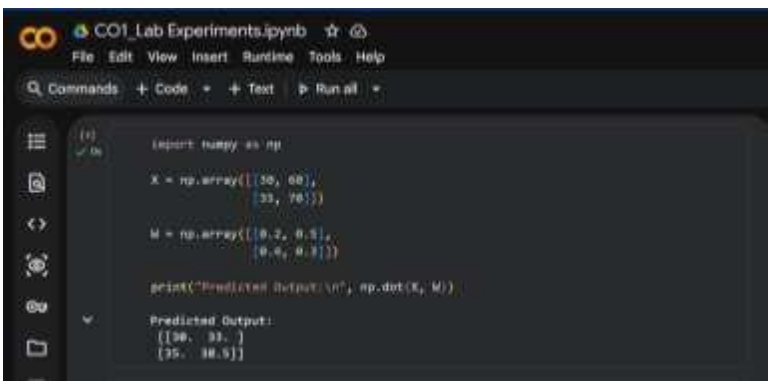
Procedure:

1. Import the NumPy library.
2. Define the input matrix and weight matrix.
3. Multiply the two matrices using NumPy.
4. Display the predicted output matrix.

Code:

```
import numpy as np
X = np.array([[30, 60],
              [35, 70]])
W = np.array([[0.2, 0.5],
              [0.4, 0.3]])
print("Predicted Output:\n", np.dot(X, W))
```

Output:



Result:

The matrix multiplication was performed successfully, producing the output matrix **[[30, 33], [35, 38.5]]**.

4.A handwriting recognition model adjusts its weights during training to reduce prediction errors. The current weight vector is W=[0.7,0.5,0.9], the gradient vector is G=[0.2,0.1,0.3], and the learning rate is 0.1.

Write a Python program to update the weights using the gradient descent rule and display the updated weight vector.

Gradient Descent Weight Update

Aim:

To update the weight vector using the gradient descent rule.

Procedure:

1. Import the NumPy library.
2. Define the weight vector and gradient vector.
3. Set the learning rate to 0.1.
4. Update and display the weights using gradient descent.

Code:

```
import numpy as np
W = np.array([0.7, 0.5, 0.9])
G = np.array([0.2, 0.1, 0.3])
lr = 0.1
W = W - lr * G
print("Updated Weights:",
W)
```

Output:



Result:

The updated weight vector was obtained as **[0.68, 0.49, 0.87]**.

5. A recommendation system represents two movies using numerical feature vectors. The first movie is represented by A=[2,4,6], and the second movie is represented by B=[1,3,5]. The system combines the feature vectors to create a combined representation and calculates the similarity score using their dot product.

Write a Python program to perform vector addition and calculate the dot product of the two movie feature vectors. Display both results.

Vector Addition and Dot Product

Aim:

To perform vector addition and calculate the dot product of two movie feature vectors.

Procedure:

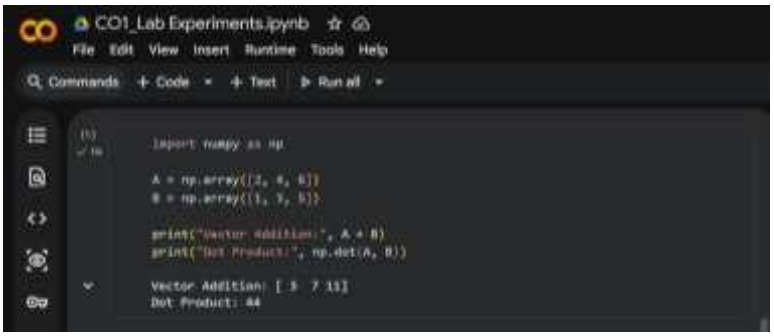
1. Import the NumPy library.
2. Define the two movie feature vectors.
3. Perform vector addition to obtain the combined representation.
4. Calculate and display the dot product.

Code:

import numpy as np

```
A = np.array([2, 4, 6])
B = np.array([1, 3, 5])
print("Vector Addition:", A + B)
print("Dot Product:", np.dot(A, B))
```

Code:



Result:

Vector addition produced **[3, 7, 11]**, and the dot product was calculated as **44**.